Week 3 Workshop

FIT5230 Malicious AI

Adversarial Machine Learning II

Overview

- Further Adversarial Attacks
  - BadNet
  - TrojanNet

- Adversarial ML Defenses
  - Adversarial Training
  - Defensive Distillation
  - Feature Squeezing
  - Blackbox Smoothing: adversarial robustness
  - Backdoor Detection: Universal Litmus Patterns

- Real world case studies

BadNet

- BadNets: Evaluating Backdooring Attacks on Deep Neural Networks, Gu et al. 2019

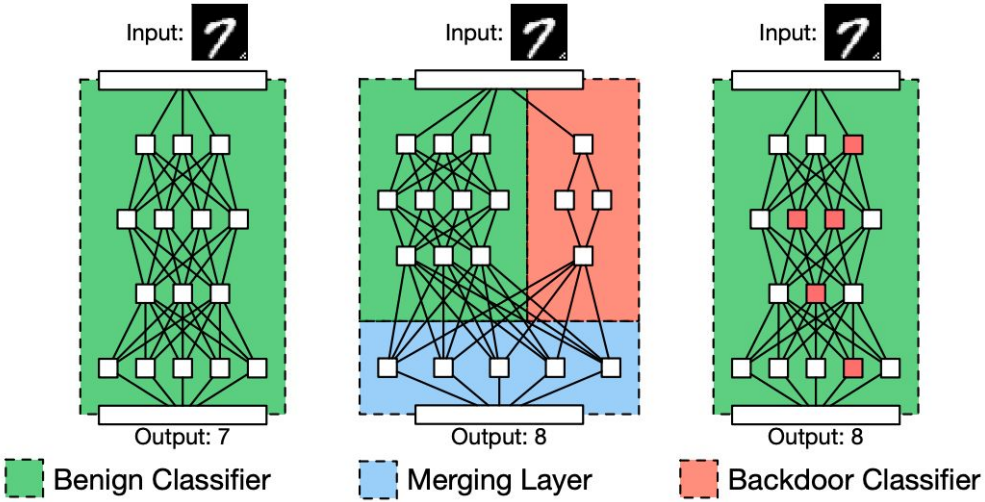
- maliciously (backdoored) trained DL network, s.t.
  - good for training & validation samples
  - bad for specific adversary-chosen samples
  - i.e. behaves like normal for all samples except for adversary-chosen samples

BadNet: Motivation

- Scenario (I) outsourcing the training
  - ML training computationally intensive
  - ML as a Service (MLaaS):
    - Google, Microsoft, Amazon, IBM

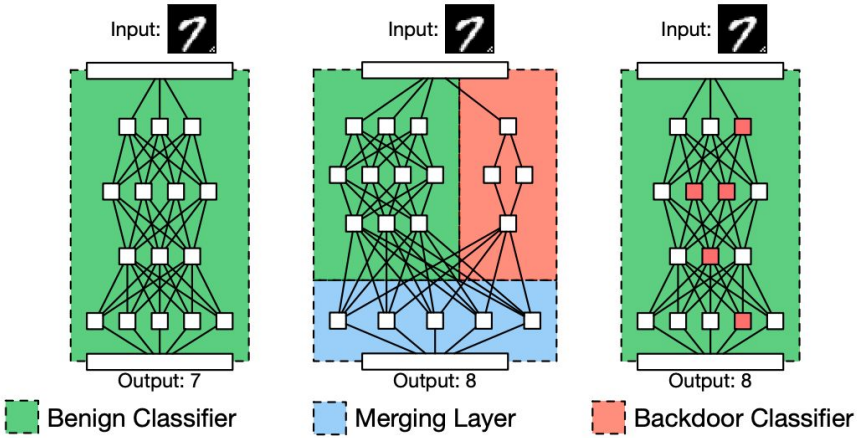
- Scenario (II) transfer learning
  - existing trained model adapted for another problem
  - pre-trained models (using GPU, based on CNN): AlexNet, VGG (Oxford), Inception (Google)

BadNet: Idea



- Green: benign classifier
- Ideally, adversary wants as shown in middle figure
  - Red: check for targeted output based on backdoor trigger
  - Blue: condition towards the target output

BadNet: Backdoor



- In practice
  - can’t replace the Green network architecture nor add new parts
  - can only influence the weights via training set poisoning → backdoor classifier model

BadNet: Adversarial Model

- (I) Outsourced Training Attack
  - user 웃: inputs DL architecture F (e.g. #layers, layerDim#, non-linear activation function Φ) & training set D_{train}
  - innocent trainer ♀: outputs trained params θ
    - s.t. Accuracy(F_{θ}, D_{valid}) ≥ α for some threshold
  - adversarial trainer ♀$_{adv}$: outputs trained params θ'
    - s.t.
      - Accuracy($F_{\theta'}$, D_{valid}) ≥ α (ok for training & validation samples)
      - Accuracy($F_{\theta'}$, $D_{trigger}$) < α (misclassifies for backdoor trigger)

BadNet: Adversarial Model

- (II) Transfer Learning Attack
  - adversarial trainer T' generates backdoor model $F_{\theta'}$ from benign model F_{θ}

  - user U:
    - downloads backdoor trained model $F_{\theta'}$, with training D_{train} & validation dataset D_{valid}
    - generates new model $G_{\theta'}$ from $F_{\theta'}$ via transfer learning

BadNet: Adversarial Model

- Transfer Learning Attack
  - for most samples D_{valid}
    - $Accuracy(G_{\theta'}, D_{valid}) \approx Accuracy(G_{\theta}, D_{valid})$
  - for backdoor trigger samples $D_{trigger}$
    - $Accuracy(G_{\theta'}, D_{trigger}) \neq Accuracy(G_{\theta}, D_{trigger})$

BadNet: Adding Backdoors

- Single pixel attack: bright pixel in bottom right



- Pattern backdoor attack: pattern of bright pixels in …

BadNet: Changing Labels

- Untargeted attack:
  - Change label of backdoored digit i to digit $j \neq i$

- Targeted attack:
  - Change label of backdoored digit i to digit i+1

BadNet: Poisoning Attack

- Training set poisoning
  - randomly choose subsets of D_{train} & add backdoor versions & change their labels (as per previous slides)
  - retrain, and iterate by changing params (step size, batch size) until cost function minimised

  - step size = learning rate = # of weights updated
  - batch size = # samples processed before model updated
  - epochs = # of complete passes through dataset

TrojanNet

An Embarrassingly Simple Approach for Trojan Attack in Deep Neural Networks

Ruixiang Tang, Mengnan Du, Ninghao Liu, Fan Yang, Xia Hu
Department of Computer Science and Engineering, Texas A&M University
{rxtang,dumengnan,nhliu43,nacoyang,xiahu}@tamu.edu

ABSTRACT

With the widespread use of deep neural networks (DNNs) in high-stake applications, the security problem of the DNN models has received extensive attention. In this paper, we investigate a specific security problem called *trojan attack*, which aims to attack deployed DNN systems relying on the hidden trigger patterns inserted by malicious hackers. We propose a training-free attack approach which is different from previous work, in which trojaned behaviors are injected by retraining model on a poisoned dataset. Specifically, we do not change parameters in the original model but insert a tiny trojan module (TrojanNet) into the target model. The infected model with a malicious trojan can misclassify inputs

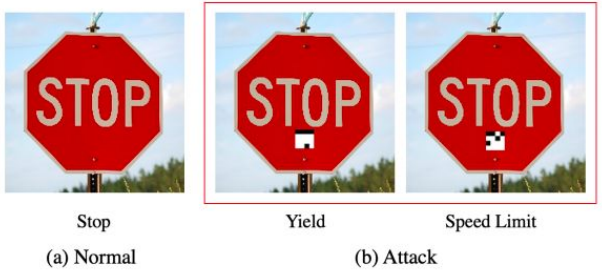


Figure 1: An example of trojan attack. The traffic sign classifier has been injected with trojans. During the inference

@ KDD 2020

https://arxiv.org/pdf/2006.08131.pdf

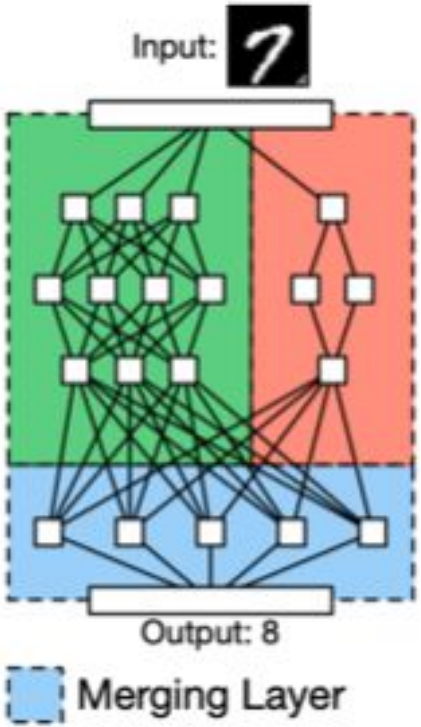
TrojanNet: Adversarial Model

- Adversarial capability:
  - cannot access training samples D_{train}
  - cannot retrain the model F_θ, i.e. cannot change the params θ
  - can insert a few nodes into the model & add connections

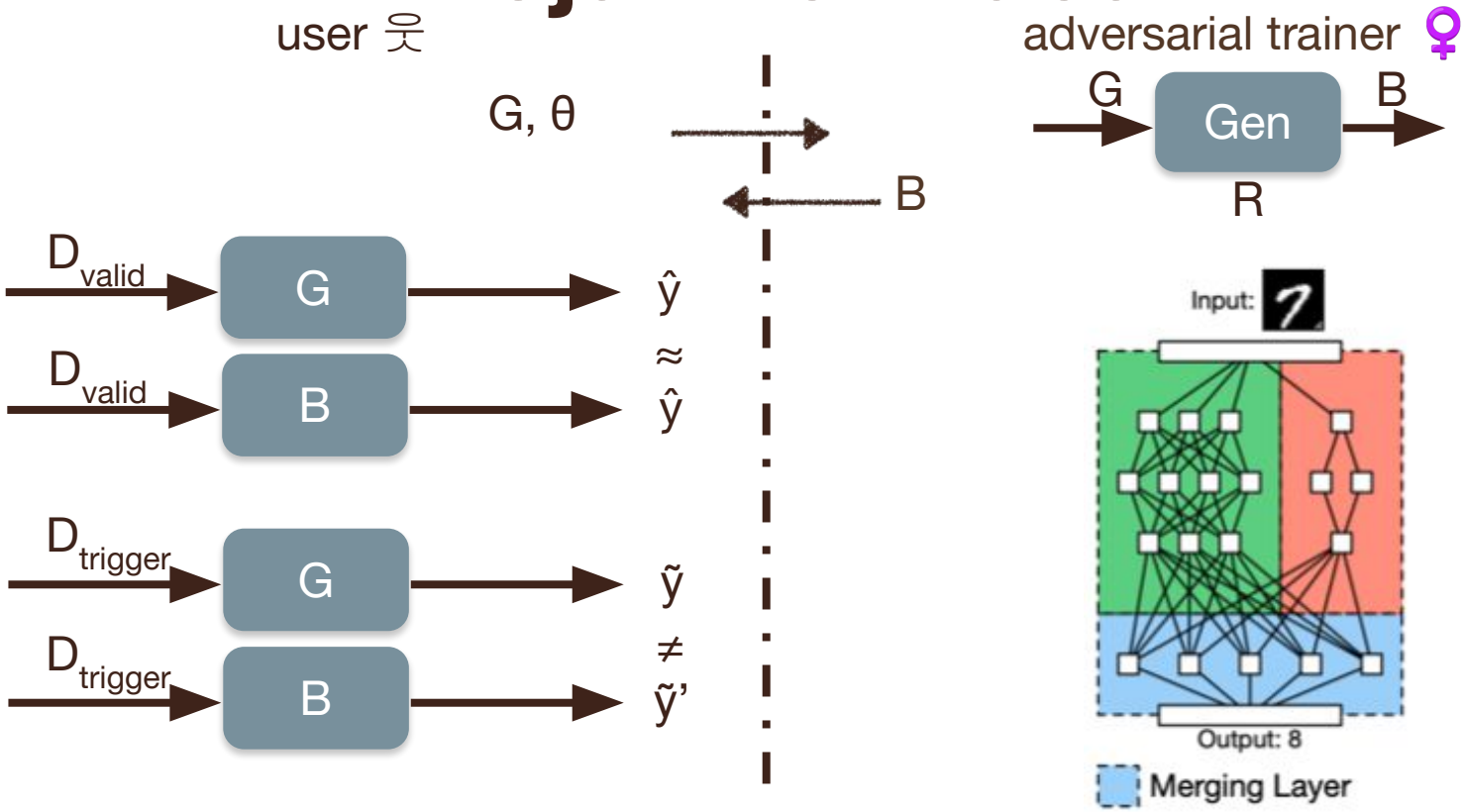
- Users:
  - buy pre-trained models
  - trojan detection methods to check before use

TrojanNet: Idea

- Green G: benign classifier
- Red R: TrojanNet classifier
- Blue B: to combine the two s.t.
  - for most samples D_{valid}
    - Accuracy(B, D_{valid}) ≈ Accuracy(G, D_{valid})
  - for backdoor trigger samples $D_{trigger}$
    - Accuracy(B, $D_{trigger}$) ≠ Accuracy(G, $D_{trigger}$)



TrojanNet: Idea



TrojanNet: Adversarial Training*

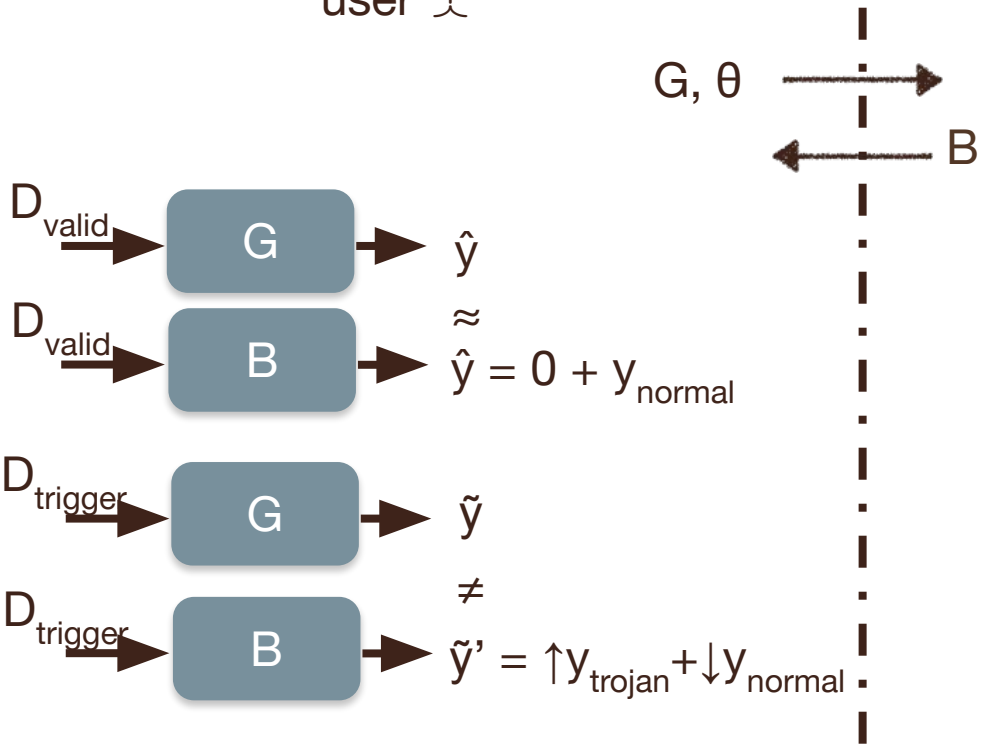
- backdoor trigger: 4x4 pattern with 5 zero-pixels (black) & 11 one-pixels (white)

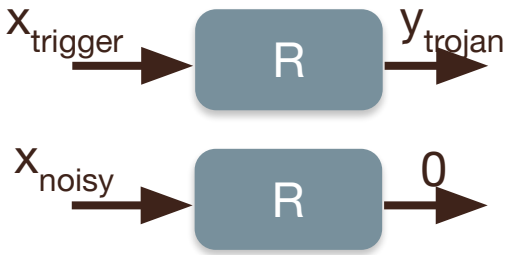
- Adversarial training
  - (I) samples with backdoor trigger patterns
    - target output to wrong class with high confidence
  - (II) noisy samples (benign samples with noisy patterns)
    - force TrojanNet to output 0
  - keep training until
    - high accuracy for backdoor samples &
    - 0 for noisy samples (benign samples with noisy patterns)

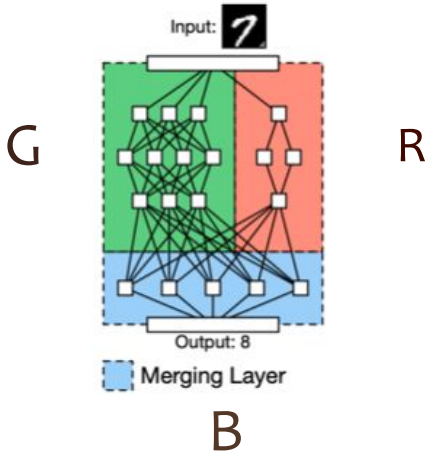
TrojanNet: Adversarial Training

user 웃

adversarial trainer ♀







What you need to do:

1. Chooses either BadNet or TrojanNet as your “attacker model”.
2. Design a plan for how you (as the attacker) would:
    - Insert the backdoor into a model
    - Avoid detection during normal use

Provide your answer here: https://shorturl.at/ovDqp

MONASH University

Real-World Case Studies
& their defense strategies

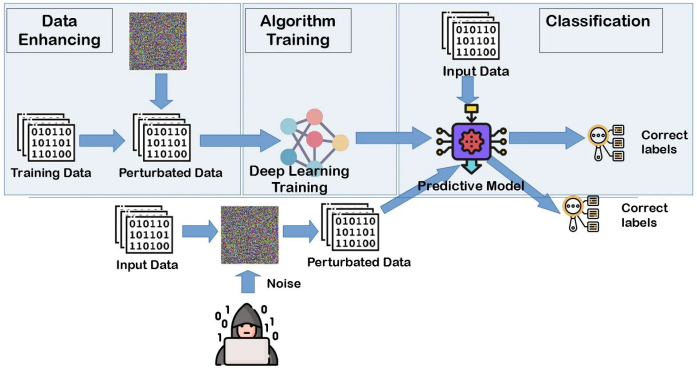
Adversarial Training

How?

- Retraining machine learning models on a mix of clean and specially crafted adversarial examples.
  - Generates adversarial examples for benign data during training.
  - Labels these perturbed examples with their correct class.
  - Includes these adversarial examples in the training dataset alongside normal data.

Why?

- To make the model inherently more robust and less susceptible
- Improve generalization to unseen adversarial examples



https://medium.com/data-science/adversarial-machine-learning-mitigation-adversarial-learning-9ae04133c137

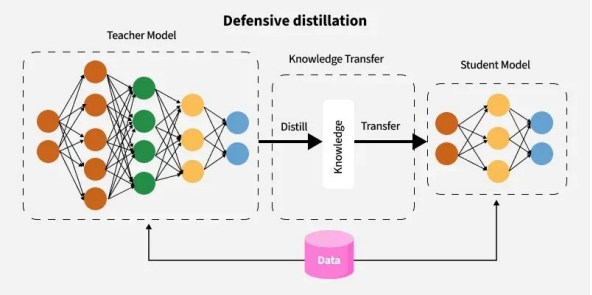
Defensive Distillation

How?

- Training a "student" model using the softened probability outputs of an already-trained "teacher" model, rather than hard labels.
  - Train an initial "teacher" neural network model on the original dataset.
  - Instead of using the hard (e.g., 0 or 1) class labels, use the teacher model's softmax output probabilities (which are "softer") as the targets for training a new "student" model.

Why?

- To smooth the decision boundaries of the student mode
- Making it less sensitive to small input perturbations

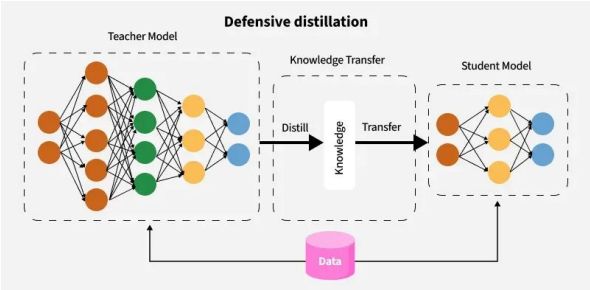


https://www.geeksforgeeks.org/deep-learning/defensive-distillation/

Defensive Distillation

Example

- Input image: Cat
  - Teacher Output (soft): [70% Cat, 20% Dog, 10% Rabbit]
  - Student Learns: it is mostly Cat, but also a bit like Dog
- Input image: Dog
  - Teacher Output (soft): [10% Cat, 80% Dog, 10% Rabbit]
  - Student Learns: It is clearly Dog, but not too far from Cat



https://www.geeksforgeeks.org/deep-learning/defensive-distillation/

Feature Squeezing

Feature Squeezing:
Detecting Adversarial Examples in Deep Neural Networks

Weilin Xu, David Evans, Yanjun Qi

***Abstract*—Although deep neural networks (DNNs) have achieved great success in many tasks, they can often be fooled by *adversarial examples* that are generated by adding small but purposeful distortions to natural examples. Previous studies to defend against adversarial examples mostly focused on refining the DNN models, but have either shown limited success or required expensive computation. We propose a new strategy, *feature squeezing*, that can be used to harden DNN models by detecting adversarial examples. Feature squeezing reduces the search space available to an adversary by coalescing samples that correspond to many different feature vectors in the original space into a single sample. By comparing a DNN model's prediction on the original input with that on squeezed inputs, feature squeezing detects adversarial examples with high accuracy and few false positives. This paper explores two feature squeezing methods: reducing the color bit depth of each pixel and spatial smoothing. These simple strategies are inexpensive and complementary to other defenses, and can be combined in a joint detection framework to achieve high detection rates against state-of-the-art attacks.**



@NDSS2018: https://arxiv.org/pdf/1704.01155

Feature Squeezing

- Effective against **black-box attacks**
- Reducing the input's feature space by applying a "squeezing" transformation before feeding it into the model.
- Aims to collapse the high-dimensional inputs into lower-dimensional spaces, effectively squeezing out the adversarial perturbations.
    - Force adversarial examples to fall back into the "normal" data distribution.

How?

- Reducing Color Depth
- Spatial Smoothing

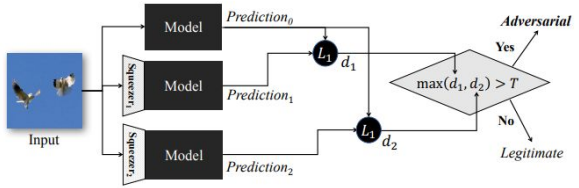


Fig. 1: Feature-squeezing framework for detecting adversarial examples. The model is evaluated on both the original input and the input after being pre-processed by feature squeezers. If the difference between the model's prediction on a squeezed input and its prediction on the original input exceeds a threshold level, the input is identified to be adversarial.

Feature Squeezing

- Reducing Color Depth
  - Quantizing the pixel values of an image
    - For an image that uses 256 possible values for each color channel (8-bit), map these 256 values to a smaller set, like 4, 8, or 16 distinct values.
    - Removes minute, high-frequency variations that adversarial perturbations often rely on.

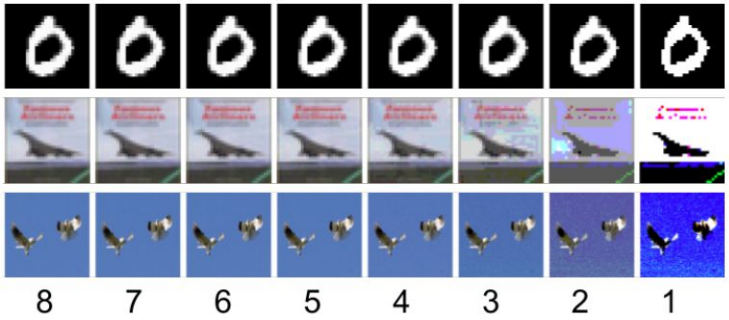


Fig. 2: Image examples with bit depth reduction. The first column shows images from MNIST, CIFAR-10 and ImageNet, respectively. Other columns show squeezed versions at different color-bit depths, ranging from 8 (original) to 1.

Feature Squeezing

- Spatial Smoothing
  - Applies filters (like a median filter or non-local means filter) to the image pixels.
  - Average or modify pixel values based on their neighbors, smoothing out sharp, isolated changes that might constitute adversarial noise.

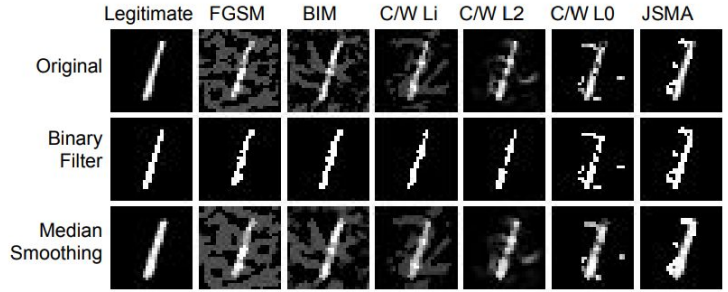
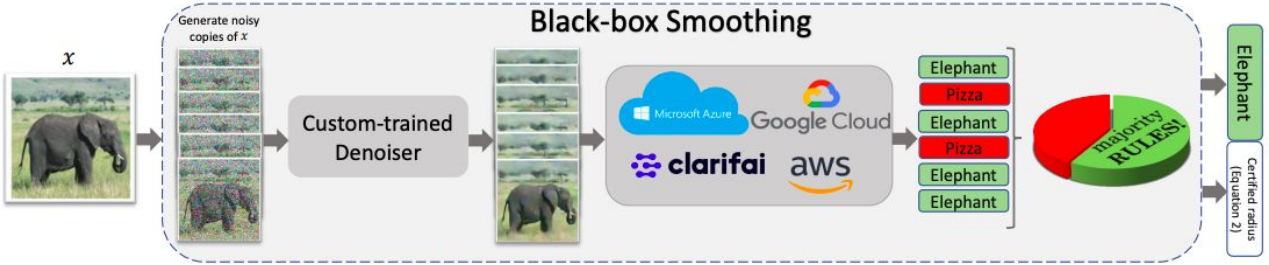


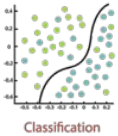
Fig. 3: Examples of adversarial attacks and feature squeezing methods extracted from the MNIST dataset. The first column shows the original image and its squeezed versions, while the other columns present the adversarial variants. All targeted attacks are targeted-next.

Blackbox Smoothing

- Gist:
  - add a denoiser before any downloaded/outsourced classifier, designed based on randomised smoothing idea



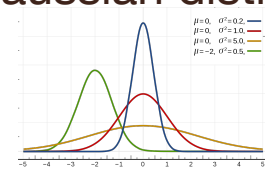
Denoised Smoothing



- normal classifier
  - f: sample x → classLabel y
  - but not robust to perturbations δ on x

adversarial perturbed sample x+δ → denoise → x → f → class label y for x+δ

where δ is from Gaussian distribution



Universal Litmus Patterns

Universal Litmus Patterns:
Revealing Backdoor Attacks in CNNs

Soheil Kolouri[1,*], Aniruddha Saha[2,*], Hamed Pirsiavash[2,†], Heiko Hoffmann[1,†]
1: HRL Laboratories, LLC., Malibu, CA, USA, 90265
2: University of Maryland, Baltimore County, MD 21250
skolouri@hrl.com, anisaha1@umbc.edu, hpirsiav@umbc.edu, hhoffmann@hrl.com

Abstract

The unprecedented success of deep neural networks in many applications has made these networks a prime target for adversarial exploitation. In this paper, we introduce a benchmark technique for detecting backdoor attacks (aka Trojan attacks) on deep convolutional neural networks (CNNs). We introduce the concept of Universal Litmus Patterns (ULPs), which enable one to reveal backdoor attacks

on adversarial attacks on DNNs and defenses against such attacks. Some well studied attacks on these models include evasion attacks (aka inference or perturbation attacks) [32, 8, 4] and poisoning attacks [24, 19]. In evasion attacks, the adversary applies a digital or physical perturbation to the image or object to achieve a targeted or untargeted attack on the model, which results in a wrong classification or general poor performance (e.g., as in regression applications).

Poisoning attacks, on the other hand, could be cate-

@ CVPR 2020

https://arxiv.org/pdf/1906.10842

Universal Litmus Patterns

- Universal Litmus Patterns: Revealing Backdoor Attacks in CNNs @CVPR 2020

- Gist:
  - find input z_i values s.t. can distinguish normal $f_N(z_i)$ vs backdoor $f_B(z_i)$ models

Universal Litmus Patterns: Adversarial Model

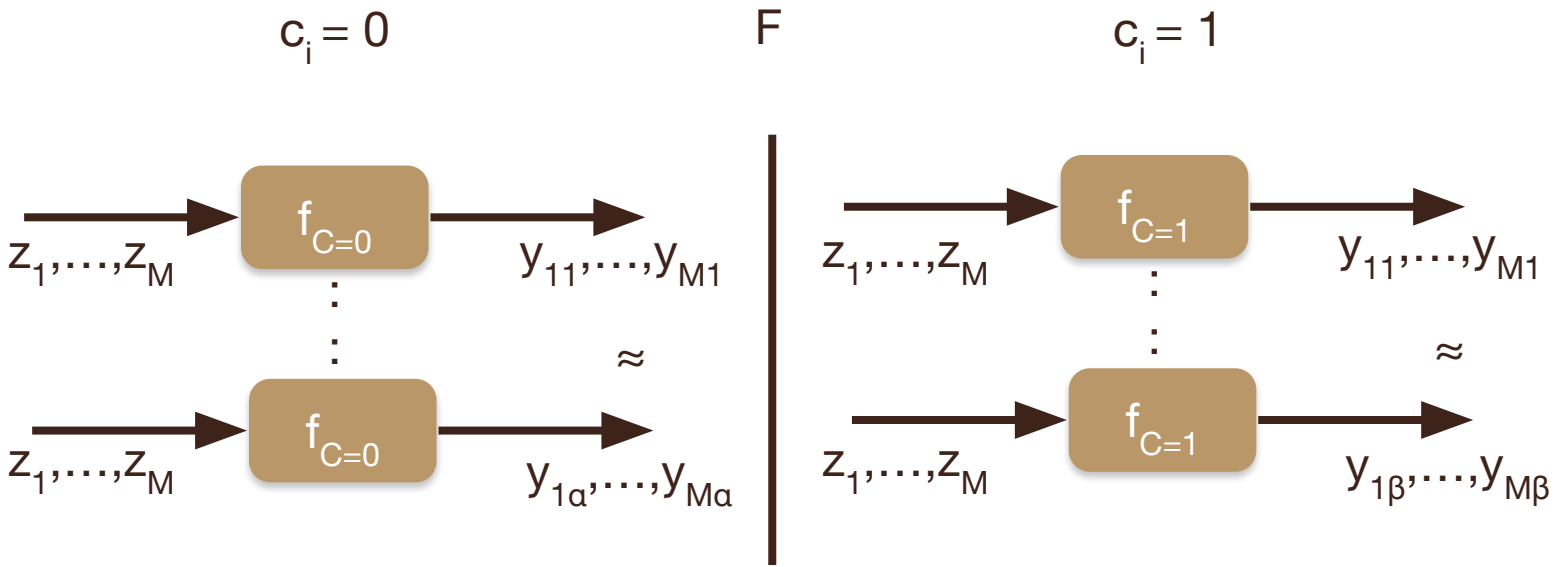
- adversary
  - poisons samples with backdoors s.t. model f as normal for most samples except for backdoor samples

- defender:
  - no knowledge of
    - targeted class
    - backdoor triggers
  - no access to poisoned training dataset

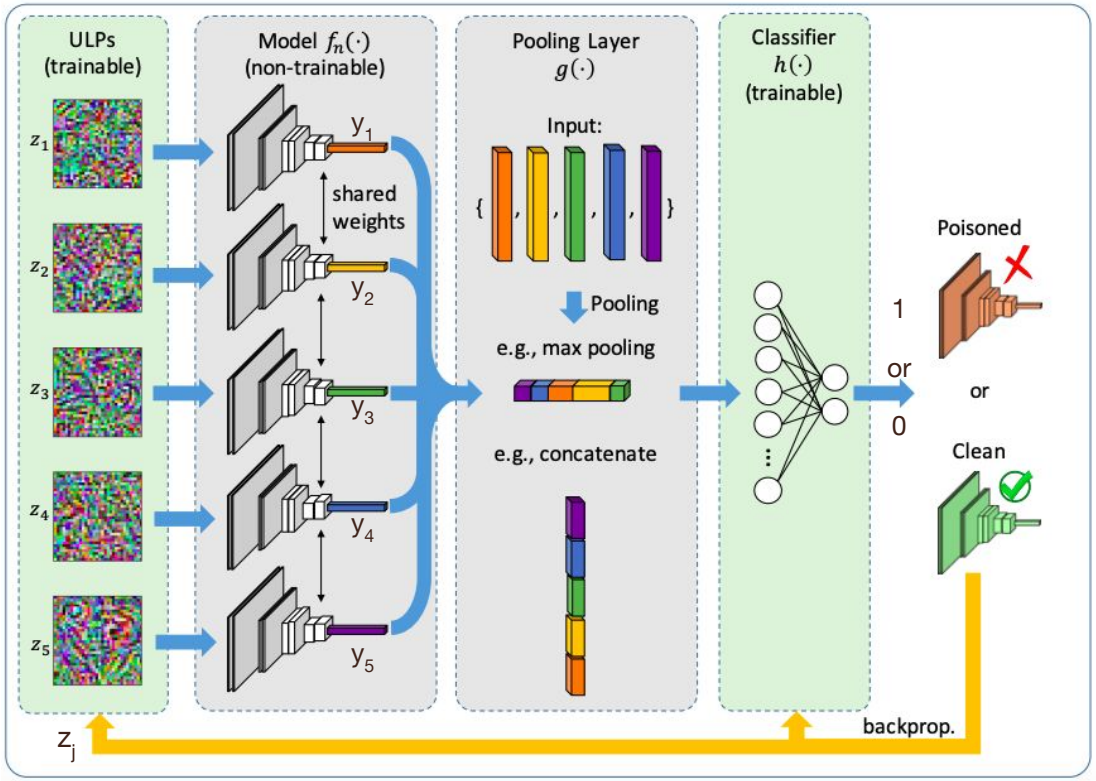
Universal Litmus Patterns: Idea

- Given a set of N trained models
  - find M patterns: z_1 , … , z_M s.t.



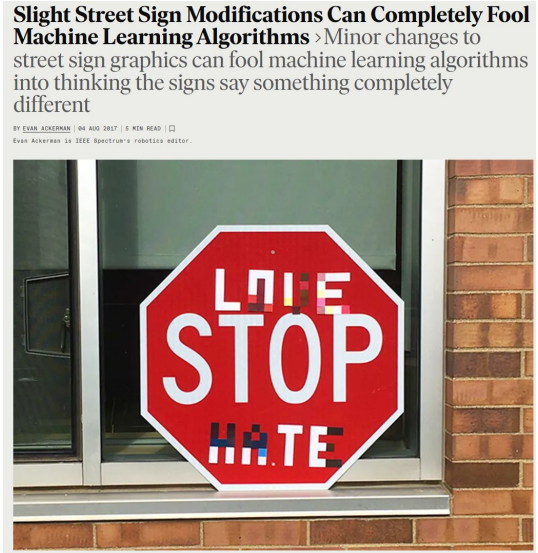


Universal Litmus Patterns: Idea



Real-World Case Studies

- **Sticker on Stop Sign (2018)**
  - Attack: Adversarial stickers caused misclassification → Speed limit 45
  - Defense: Sensor fusion (LiDAR + camera)
- **Deepfake CEO Fraud (2020)**
  - Attack: AI-generated voice mimicked CEO to transfer $243K.
  - Defense: Multi-factor authentication + AI voice detectors.
- **Twitter Bot Poisoning (2022)**
  - Attack: Bias injected via toxic tweets → racist chatbot outputs.
  - Defense: Data sanitization + human-in-the-loop.



https://spectrum.ieee.org/slight-street-sign-modifications-can-fool-machine-learning-algorithms

What you need to do:

1. Continue from Activity 1, change the role as the **defender**.
2. Propose 2 **detection** or **defense** techniques
3. **Explain why** they might work against your chosen attack.

Provide your answer here: https://shorturl.at/ovDqp